

Trabajo Práctico Nro. 2

Arquitectura REST con Spring Boot

Análisis y Diseño de Sistemas II

September 9, 2026

Objetivos

El objetivo de este trabajo práctico es desarrollar una API REST con Spring Boot para gestionar diferentes funcionalidades de una tienda de tecnología.

Todos los ejercicios estarán relacionados con TechStore, un pequeño sistema de gestión de una tienda que trabaja con productos, categorías, clientes, pedidos y ventas.

A lo largo de los ejercicios se incorporarán distintos conceptos de desarrollo de APIs REST, como:

- creación de endpoints con Spring Boot;
- uso de diferentes métodos HTTP (GET, POST, PUT y DELETE);
- recepción de datos mediante @RequestBody, @PathVariable y @RequestParam;
- utilización de DTOs para representar los datos que entran y salen de la API;
- validación de datos recibidos;
- manejo de errores y respuestas HTTP;
- separación de responsabilidades entre controlador, servicio y otras capas de la aplicación;
- documentación y prueba de los endpoints mediante Swagger/OpenAPI.

Todos los ejercicios giran en torno a **TechStore**, un pequeño sistema de gestión de una tienda de tecnología: productos, categorías, clientes y pedidos. Van a ir construyendo distintos módulos de la misma aplicación.

Consignas generales

Los siguientes requisitos deben ser aplicados a todos los ejercicios.

1. **Formato de respuesta estándar.** Toda respuesta, sin excepción (éxito o error), debe tener esta forma:

```
{
  "status": 200,
  "message": "Operacion realizada con exito",
  "data": {}
}
```

- **status:** código HTTP como número (200, 201, 400, 404, 500, etc.)

- **message:** mensaje descriptivo, en español, de lo que pasó.
- **data:** el contenido real de la respuesta. Puede ser un objeto, una lista, o `null` en errores.

Se recomienda armar una clase genérica `ApiResponse<T>` reutilizable y un `@ControllerAdvice` con `@ExceptionHandler` para que los errores (validación, no encontrado, error interno) también respeten este formato automáticamente, sin tener que armarlo a mano en cada controller.

2. **Documentación con Swagger.** Todos los endpoints deben estar documentados con `springdoc-openapi` (`@Operation`, `@ApiResponse`, `@Parameter`, `@Schema` donde corresponda). La corrección se va a probar **desde Swagger UI** (`/swagger-ui.html`).
3. **Códigos de estado correctos.** 200/201 en éxito, 400 en datos inválidos, 404 en recurso no encontrado, 500 solo para errores no controlados.
4. **Paquetes sugeridos:** `controller`, `service`, `repository`, `dto`, `model/entity`, `exception`.
5. **Convenciones de código**

El código deberá respetar las convenciones de nombres habituales de Java. En particular:

- Las **clases e interfaces** deberán utilizar `UpperCamelCase`, por ejemplo: `ProductoDTO`.
- Los **métodos y variables** deberán utilizar `lowerCamelCase`, por ejemplo: `buscarProductos` o `precioMaximo`.
- Las **constantes** deberán utilizar `UPPER_SNAKE_CASE`, por ejemplo: `MAX_STOCK`.
- Los **paquetes** deberán escribirse utilizando letras minúsculas.
- Los nombres deberán ser **claros, descriptivos y consistentes**, evitando abreviaturas innecesarias.

Como referencia, consultar la guía [Java Naming Conventions](#) de Baeldung.

1 Ejercicio 1 — Procesamiento de ventas

La tienda recibe lotes de ventas desde otro sistema y los quiere analizar. En cada petición recibe una lista de ventas en formato JSON y se devuelve un conjunto de estadísticas calculadas.

La información recibida no debe guardarse en una base de datos. Simplemente se procesa en memoria y se devuelve el resultado.

Cada venta tiene:

```
{
  "producto": "Mouse inalambrico",
  "cantidad": 3,
  "precioUnitario": 4500.0
}
```

Para recibir la lista de ventas deberán utilizar `@RequestBody` y `@Valid`.

Endpoint 1: POST `/api/ventas/estadisticas`

Recibe una **lista** de `VentaDTO` y devuelve:

- **totalFacturado**: suma de ($\text{cantidad} \times \text{precioUnitario}$) de todas las ventas.
- **cantidadVentas**: cantidad de ventas recibidas.
- **ticketPromedio**: $\text{totalFacturado} / \text{cantidadVentas}$.
- **ventaMayor** y **ventaMenor**: el objeto venta (con su importe) de mayor y menor monto.
- **productoMasVendido**: el nombre del producto con mayor cantidad acumulada (sumando cantidad si se repite el producto en la lista).

Validaciones requeridas (con Bean Validation, `@Valid`):

- **producto**: no vacío.
- **cantidad**: entero positivo (mayor a 0).
- **precioUnitario**: positivo, mayor a 0.
- La lista no puede venir vacía → 400 con mensaje claro.

Si alguno de los elementos de la lista no cumple con las validaciones, el endpoint debe responder con HTTP 400 (Bad Request) e informar claramente:

- la posición del elemento que contiene el error;
- el campo que no es válido;
- el motivo del error.

Para manejar estos errores se deberá utilizar un `@ExceptionHandler` para `MethodArgumentNotValidException`.

Endpoint 2: POST /api/ventas/aplicar-descuento

Se recibe:

- la misma lista de VentaDTO mediante @RequestBody;
- un porcentaje de descuento mediante @RequestParam.

Para cada venta se debe calcular su monto con descuento y devolverlo como un campo adicional llamado `montoConDescuento`.

Además, se debe calcular el `totalConDescuento`, que corresponde a la suma de los montos de todas las ventas después de aplicar el descuento.

La respuesta debe incluir la lista de ventas con el nuevo campo `montoConDescuento` y el `totalConDescuento` general.

Validación del porcentaje

El parámetro porcentaje debe ser un valor entre 0 y 100, inclusive.

- 0: no se aplica descuento.
- 10: se aplica un descuento del 10
- 100: se aplica un descuento del 100
- Un valor menor que 0 o mayor que 100: se debe devolver HTTP 400 (Bad Request) con un mensaje claro indicando que el porcentaje no es válido.

2 Ejercicio 2 – Gestión y búsqueda de productos en memoria

Módulo: Catálogo de productos En este ejercicio se construirá un pequeño catálogo de productos que se mantendrá en memoria, sin utilizar una base de datos.

La aplicación deberá tener una colección `List<Producto>` dentro de un `@Service`. Al iniciar la aplicación, la colección deberá cargarse con al menos 8 productos de ejemplo.

Cada producto tendrá, como mínimo, los siguientes datos:

- id
- nombre
- categoria
- precio
- stock

El objetivo es implementar diferentes operaciones sobre esta colección: consultar productos, buscar y filtrar, ordenar, agregar, modificar y eliminar.

GET /api/catalogo

Devuelve todos los productos disponibles en el catálogo.

GET /api/catalogo/buscar?

Permite buscar productos utilizando los siguientes parámetros de consulta:
`/api/catalogo/buscar?categoria=Perifericos&precioMin=1000&precioMax=10000`
Todos los parámetros son opcionales y pueden combinarse.

- **categoria:** filtra los productos que pertenecen a esa categoría.
- **precioMin:** devuelve productos cuyo precio sea mayor o igual a ese valor.
- **precioMax:** devuelve productos cuyo precio sea menor o igual a ese valor.

Si se envía más de un parámetro, se deben aplicar todos los filtros al mismo tiempo.

Si no se envía ningún parámetro, se deben devolver todos los productos.

Importante: el filtrado deberá realizarse utilizando Streams de Java, especialmente `filter()`.

GET /api/catalogo/ordenar

Permite ordenar los productos del catálogo utilizando parámetros de consulta.

Ejemplo:

`/api/catalogo/ordenar?criterio=precio&orden=desc`

El parámetro `criterio` puede tomar los siguientes valores:

- `precio`
- `nombre`

El parámetro `orden` puede ser:

- `asc`: orden ascendente.
- `desc`: orden descendente.

Si no se envía orden, se debe utilizar **asc** como valor predeterminado.

Importante: para realizar el ordenamiento deberán utilizar **Streams** de Java y **Comparator**.

POST /api/catalogo

Recibe un nuevo producto mediante `@RequestBody`.

Ejemplo:

```
{
  "nombre": "Teclado",
  "categoria": "Perifericos",
  "precio": 25000,
  "stock": 10
}
```

El producto debe agregarse a la colección en memoria.

Validar que:

- nombre no esté vacío.
- precio sea mayor que 0.
- stock sea mayor o igual que 0.

Si los datos son correctos, se debe devolver HTTP 201 (Created).

PUT /api/catalogo/{id}/stock?cantidad=5

Permite modificar el stock de un producto. El parámetro **cantidad** indica cuánto se debe modificar el stock actual:

- Un valor positivo aumenta el stock.
- Un valor negativo disminuye el stock.

El stock nunca puede quedar por debajo de 0.

Si la modificación provocaría un stock negativo, se debe devolver HTTP 400 (Bad Request).

Si no existe un producto con el **id** indicado, se debe devolver HTTP 404 (Not Found).

DELETE /api/catalogo/{id}

Elimina de la colección el producto que tenga el **id** indicado.

- Si el producto existe, se elimina correctamente.
- Si no existe, se debe devolver HTTP 404 (Not Found).

3 Ejercicio 3 — Consumo de una API externa

Módulo: Conversor de divisas

En este ejercicio se incorporará una nueva funcionalidad a la aplicación: un conversor de divisas.

A diferencia de los ejercicios anteriores, en este caso la aplicación deberá obtener información desde un servicio externo. Para ello se utilizará la API pública y gratuita Frankfurter, que permite consultar tasas de cambio entre diferentes monedas y no requiere una API key.

La API utiliza la siguiente URL:

<https://api.frankfurter.app/latest?amount={monto}&from={origen}&to={destino}>

El endpoint recibirá los siguientes parámetros mediante `@RequestParam`:

- **monto**: cantidad de dinero que se desea convertir.
- **origen**: código de la moneda de origen.
- **destino**: código de la moneda de destino.

Ejemplo: `GET /api/divisas/convertir?monto=100&origen=USD&destino=ARS`

GET /api/divisas/convertir?monto=100&origen=USD&destino=ARS

1. Validar los datos recibidos.

- **monto** debe ser mayor que 0.
- **origen** debe ser un código de moneda de 3 letras.
- **destino** debe ser un código de moneda de 3 letras.

Se pueden utilizar códigos ISO como:

USD
ARS
EUR
BRL

Si algún dato no es válido, se debe devolver **HTTP 400 (Bad Request)** con un mensaje claro.

2. Consultar la API externa.

La aplicación deberá realizar una petición HTTP a Frankfurter utilizando los datos recibidos.

Para realizar la llamada se deberá utilizar `RestClient` de Spring. También puede utilizarse `RestTemplate` si así se indica en clase.

3. Procesar la respuesta.

La respuesta recibida desde Frankfurter contiene información que no necesariamente debe ser enviada al cliente.

La aplicación deberá **parsear la respuesta y construir su propia respuesta**, devolviendo solamente los datos relevantes para nuestra aplicación.

La respuesta deberá tener una estructura similar a:

```
{
  "montoOriginal": 100,
  "monedaOrigen": "USD",
  "monedaDestino": "ARS",
  "tasaCambio": 1234.56,
  "montoConvertido": 123456.0,
  "fecha": "2026-09-02"
}
```

Manejo de errores

La aplicación debe contemplar posibles problemas al comunicarse con el servicio externo. Por ejemplo:

- Frankfurter no responde.
- Se produce un timeout.
- La API externa devuelve un error HTTP.
- Se solicita una moneda que no existe.

Estos errores deben ser capturados y tratados por la API.

Según el tipo de error, se deberá devolver una respuesta adecuada, por ejemplo:

- HTTP 400 (Bad Request) cuando los datos enviados por el cliente sean incorrectos.
- HTTP 502 (Bad Gateway) cuando la API no pueda obtener correctamente la información del servicio externo.

La respuesta debe incluir un mensaje claro que permita entender qué ocurrió.

Consideraciones técnicas

La llamada a Frankfurter puede realizarse utilizando, por ejemplo:

`RestClient.builder().build()` o mediante un `RestClient` configurado como `bean` e inyectado en el servicio.

El procesamiento de la llamada deberá realizarse preferentemente en una capa de `Service`, manteniendo en el `Controller` la responsabilidad de recibir los parámetros y devolver la respuesta HTTP.